JavaScript Fundamentals – Operators, Events, Control Flow, Loops, Arrays, Objects, and the DOM

JavaScript is the programming language of the web. Every interactive website — from a simple button that changes color to a complex web application that responds to clicks, keystrokes, and scrolling — relies on JavaScript. Unlike HTML, which defines the structure of a page, and CSS, which defines its appearance, JavaScript provides **behavior**. It allows web pages to make decisions, respond to user actions, store and manipulate data, and update the page dynamically without reloading. Learning JavaScript is essential for any student who wants to build modern web applications, and it forms the foundation for understanding how browsers execute code on the client side.

This reading material introduces the core building blocks of JavaScript. It explains how to use JavaScript operators in problem solving. It describes how event handlers and event listeners respond to user actions. It covers conditional and switch statements for decision-making, and while, do-while, and for loops for repetition. It also explains how to work with arrays and objects to store and organize data, and how to manipulate Document Object Model (DOM) nodes to create and modify web page content.

Part 1: JavaScript Operators

JavaScript, like every programming language, uses **operators** to perform operations on values and variables. Operators are the tools that allow programs to calculate, compare, assign, and combine data. Without operators, a program could only store values — it could not process them. Understanding the different categories of operators is the first step toward writing JavaScript that actually does something useful.

Categories of JavaScript Operators

Category	Purpose	Examples
Arithmetic	Perform mathematical calculations	`+`, `-`, `*`, `/`, `%`, `**`
Assignment	Assign values to variables	`=`, `+=`, `-=`, `*=`, `/=`
Comparison	Compare two values and return true or false	`==`, `===`, `!=`, `!==`, `>`, `<`, `>=`, `<=`
Logical	Combine boolean expressions	`&&` (AND), `||` (OR), `!` (NOT)
Increment / Decrement	Increase or decrease by 1	`++`, `--`
String	Concatenate strings	`+`, `` `${}` `` (template literals)
Ternary	Short form of if-else	`condition ? value1 : value2`
Type	Check or convert types	`typeof`, `instanceof`

Arithmetic Operators

Arithmetic operators are used for mathematical calculations.

Operator	Meaning	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`10 - 4`	`6`
`*`	Multiplication	`6 * 7`	`42`
`/`	Division	`20 / 4`	`5`
`%`	Modulus (remainder)	`10 % 3`	`1`

Operator	Meaning	Example	Result
`**`	Exponentiation	`2 ** 4`	`16`

Example Code:

```javascript
let price = 150;
let quantity = 3;
let total = price * quantity;      // 450
let discount = total * 0.10;       // 45
let finalPrice = total - discount; // 405

console.log("Total: " + total);         // Total: 450
console.log("Discount: " + discount);   // Discount: 45
console.log("Final: " + finalPrice);    // Final: 405
```

Assignment Operators

Assignment operators assign values to variables. Compound assignment operators combine an operation with assignment.

Operator	Meaning	Equivalent To
`=`	Assign	`x = 5`
`+=`	Add and assign	`x = x + 5`
`-=`	Subtract and assign	`x = x - 5`
`*=`	Multiply and assign	`x = x * 5`
`/=`	Divide and assign	`x = x / 5`
`%=`	Modulus and assign	`x = x % 5`

```javascript
let score = 100;
```

```
score += 20;   // score is now 120
score -= 10;   // score is now 110
score *= 2;    // score is now 220
```

Comparison Operators

Comparison operators compare two values and return a **boolean** (`true` or `false`).

Operator	Meaning	Example	Result
`==`	Equal to (value only)	`5 == "5"`	`true`
`===`	Strict equal (value and type)	`5 === "5"`	`false`
`!=`	Not equal	`5 != "5"`	`false`
`!==`	Strict not equal	`5 !== "5"`	`true`
`>`	Greater than	`10 > 5`	`true`
`<`	Less than	`10 < 5`	`false`
`>=`	Greater than or equal	`10 >= 10`	`true`
`<=`	Less than or equal	`5 <= 3`	`false`

Best practice: Use `===` and `!==` (strict comparison) rather than `==` and `!=` to avoid unexpected type coercion bugs.

Logical Operators

Logical operators combine boolean expressions.

Operator	Meaning	Example	Result
`&&`	AND — true if both are true	`true && false`	`false`
`||`	OR — true if at least one is true	`true || false`	`true`
`!`	NOT — reverses the boolean	`!true`	`false`

```javascript
let isLoggedIn = true;
let isAdmin = false;

if (isLoggedIn && isAdmin) {
  console.log("Welcome, admin.");
} else if (isLoggedIn || isAdmin) {
  console.log("Welcome, user.");
} else {
  console.log("Please log in.");
}
// Output: Welcome, user.
```

Increment and Decrement Operators

Operator	Meaning	Example
`++`	Increment by 1	`x++` or `++x`
`--`	Decrement by 1	`x--` or `--x`

The position of the operator matters: `x++` (post-increment) returns the value *before* incrementing, while `++x` (pre-increment) returns the value *after* incrementing.

String Operators and Template Literals

The `+` operator concatenates strings. Template literals (backticks) allow embedding expressions inside strings.

```javascript
let firstName = "Juan";
let lastName = "Dela Cruz";
let greeting = "Hello, " + firstName + " " + lastName + "!";

// Template literal — cleaner and easier to read
let greeting2 = `Hello, ${firstName} ${lastName}!`;

console.log(greeting2);  // Hello, Juan Dela Cruz!
```

Ternary Operator

The ternary operator is a compact `if-else` in one line.

```javascript
let age = 18;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status);  // Adult
```

Operator Precedence

JavaScript follows the standard mathematical order of operations. Use parentheses to make the intended order explicit.

Priority	Operators
1 (highest)	`()` — parentheses
2	`**` — exponentiation
3	`*`, `/`, `%`
4	`+`, `-`

Priority	Operators
5	`==`, `===`, `!=`, `!==`, `<`, `>`, `<=`, `>=`
6	`&&`
7	`||`
8 (lowest)	`=`, `+=`, `-=`, etc.

Part 2: JavaScript Events – Event Handlers and Event Listeners

Web pages become interactive when they respond to **events**. An event is an action or occurrence that happens in the browser — a click, a keystroke, a mouse movement, a page load. JavaScript allows developers to "listen" for these events and execute code when they occur. Two mechanisms enable this: **event handlers** (older, inline style) and **event listeners** (modern, recommended style). Understanding both is essential for building responsive web applications.

What Is an Event?

An **event** is any action or occurrence that the browser can detect and to which JavaScript can respond. Events are triggered by users (mouse clicks, keyboard input) or by the browser itself (page load, timer expiration).

Common JavaScript Events

Event	Triggered When
`click`	User clicks on an element
`dblclick`	User double-clicks on an element
`mouseover`	Mouse pointer moves over an element
`mouseout`	Mouse pointer moves out of an element
`keydown`	User presses a key
`keyup`	User releases a key
`submit`	A form is submitted
`change`	The value of an input changes
`load`	The page or an image finishes loading
`focus`	An element gains focus
`blur`	An element loses focus

Event Handlers (Inline and Property-Based)

An **event handler** attaches code directly to an HTML element or to a JavaScript property. There are two ways to use event handlers.

Method 1: HTML Attribute (Inline)

```
html
<button onclick="alert('Hello!')">Click Me</button>
```

Method 2: JavaScript Property

```
html
<button id="myButton">Click Me</button>

<script>
  let button = document.getElementById("myButton");
  button.onclick = function() {
    alert("Hello!");
  };
</script>
```

Limitations of event handlers:

Limitation	Explanation
Only one handler per event	If you assign `onclick` twice, the second overwrites the first
Mixes HTML and JavaScript	Inline handlers violate separation of concerns
Harder to remove	Removing a handler requires setting the property back to `null`

Event Listeners (Recommended)

An **event listener** is attached using `addEventListener()`. It is the modern, preferred approach because it allows multiple listeners on the same element and keeps JavaScript separate from HTML.

Syntax:

```
javascript
element.addEventListener("event", function);
```

Example:

```
html
<button id="myButton">Click Me</button>
```

```
<script>
  let button = document.getElementById("myButton");
  button.addEventListener("click", function() {
    alert("Hello from event listener!");
  });
</script>
```

Multiple Listeners on the Same Event

A major advantage of event listeners is that multiple listeners can be attached to the same event without overwriting each other.

```
javascript
let button = document.getElementById("myButton");

button.addEventListener("click", function() {
  console.log("First listener");
});

button.addEventListener("click", function() {
  console.log("Second listener");
});

// Both listeners run when the button is clicked.
```

Removing Event Listeners

Use `removeEventListener()` to detach a listener. The callback function must be a **named function** (not anonymous) so it can be referenced later.

```
javascript
function handleClick() {
  console.log("Clicked!");
}

button.addEventListener("click", handleClick);

// Later:
button.removeEventListener("click", handleClick);
```

The Event Object

When an event fires, JavaScript passes an **event object** to the callback. It contains details about the event, such as the target element, mouse position, and pressed keys.

```
javascript
document.addEventListener("keydown", function(event) {
  console.log("Key pressed: " + event.key);
});
```

Event Delegation

Instead of adding a listener to every child element, you can add one listener to a parent and use `event.target` to determine which child was clicked. This is called **event delegation** and improves performance when there are many elements.

```
javascript
document.getElementById("list").addEventListener("click", function(event) {
  if (event.target.tagName === "LI") {
    console.log("You clicked: " + event.target.textContent);
  }
});
```

Comparison: Event Handlers vs. Event Listeners

Feature	Event Handlers	Event Listeners
Syntax	`element.onclick = fn`	`element.addEventListener("click", fn)`
Multiple handlers per event	No	Yes
Can be removed easily	Difficult	Yes, with `removeEventListener`
Modern best practice	No	Yes
Separation of HTML and JS	Often violated (inline)	Maintained

Part 3: Control Flow – Conditional and Switch Statements

Control flow determines the order in which statements execute. Without control flow, a program would run every line of code from top to bottom, regardless of conditions. JavaScript provides two main mechanisms for branching: **conditional statements** (`if`, `else if`, `else`) and **switch statements**. These allow programs to make decisions based on data and user input.

The `if` Statement

The `if` statement executes a block of code only when a condition is true.

```javascript
let score = 85;

if (score >= 75) {
  console.log("You passed!");
}
```

The `if...else` Statement

The `else` block runs when the `if` condition is false.

```javascript
let score = 60;

if (score >= 75) {
  console.log("You passed!");
} else {
  console.log("You need to retake the exam.");
}
```

The `if...else if...else` Chain

Multiple conditions can be checked in sequence.

```
javascript
let score = 88;

if (score >= 90) {
  console.log("Outstanding");
} else if (score >= 85) {
  console.log("Very Satisfactory");
} else if (score >= 75) {
  console.log("Satisfactory");
} else {
  console.log("Failed");
}
```

Truthy and Falsy Values

In JavaScript, conditions do not always need to be booleans. Values are evaluated as either **truthy** or **falsy**.

Falsy Values	Truthy Values (examples)
`false`, `0`, `""` (empty string), `null`, `undefined`, `NaN`	Everything else, including `"0"`, `[]`, `{}`, `"false"`

```
javascript
if ("hello") {
  console.log("This runs because non-empty strings are truthy.");
}

if (0) {
  console.log("This never runs because 0 is falsy.");
}
```

The `switch` Statement

The `switch` statement is used when there are many possible values for a single expression. It compares the expression to each `case` and executes the matching block.

```
javascript
let day = "Monday";

switch (day) {
  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("Weekday");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend");
    break;
  default:
    console.log("Invalid day");
}
```

The `break` Keyword

The `break` statement exits the switch block. Without it, execution "falls through" to the next case, which is usually an unintended bug.

When to Use `if` vs. `switch`

Situation	Recommended
Testing a range of values (e.g., `score >= 75`)	`if...else`
Testing boolean conditions	`if...else`

Situation	Recommended
Testing many exact values of the same variable	`switch`
Checking complex conditions with `&&` or `||`	`if...else`

Part 4: Loops – `while`, `do...while`, and `for`

Loops allow a program to repeat a block of code multiple times without rewriting the same lines. JavaScript provides three main loops: `while`, `do...while`, and `for`. Each has a specific use case depending on whether the number of iterations is known in advance and whether the loop body must run at least once.

The `while` Loop

A `while` loop runs as long as its condition is true. The condition is checked **before** each iteration.

Syntax:

```
javascript
while (condition) {
  // code
}
```

Example — Print numbers 1 to 5:

```
javascript
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Use case: When the number of iterations is not known in advance (e.g., waiting for user input to match a condition).

The `do...while` Loop

A `do...while` loop runs the body **at least once**, then checks the condition.

Syntax:

```
javascript
do {
  // code
} while (condition);
```

Example — Ask for input until valid:

```
javascript
let number;
do {
  number = prompt("Enter a number greater than 10:");
} while (number <= 10);
```

Use case: When the body must execute at least once (e.g., input validation).

The `for` Loop

A `for` loop is used when the number of iterations is known in advance. It combines initialization, condition, and increment in one line.

Syntax:

```
javascript
for (initialization; condition; increment) {
  // code
}
```

Example — Print numbers 1 to 5:

```
javascript
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Looping Through Arrays with `for...of`

The modern `for...of` loop iterates directly over each element of an array.

```javascript
let fruits = ["apple", "banana", "mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

Looping Through Object Properties with `for...in`

The `for...in` loop iterates over the keys of an object.

```javascript
let student = { name: "Juan", age: 17, grade: "11" };

for (let key in student) {
  console.log(key + ": " + student[key]);
}
```

Infinite Loops and How to Avoid Them

An **infinite loop** never terminates because the condition never becomes false. Always ensure the loop's condition will eventually be met.

```javascript
// Infinite loop — avoid!
let i = 1;
while (i <= 5) {
  console.log(i);
  // missing i++
}
```

The `break` and `continue` Keywords

Keyword	Effect
`break`	Exits the loop entirely
`continue`	Skips the rest of the current iteration and jumps to the next

```javascript
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;
  console.log(i);  // prints 1, 2, 3, 4
}

for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i);  // prints 1, 2, 4, 5
}
```

Comparison of Loops

Loop	Condition Check	Runs at Least Once?	Best For
`while`	Before	No	Unknown iteration count
`do...while`	After	Yes	Must run at least once
`for`	Before	No	Known iteration count
`for...of`	Before	No	Iterating arrays
`for...in`	Before	No	Iterating object keys

Part 5: JavaScript Arrays

An **array** is an ordered collection of values stored in a single variable. Instead of creating ten separate variables for ten items, a single array can hold all of them under one name and each value can be accessed by its **index**. Arrays are essential for organizing data such as lists of names, scores, prices, or any group of related values.

Creating Arrays

```
javascript
let fruits = ["apple", "banana", "mango"];
let numbers = [10, 20, 30, 40];
let mixed = ["Juan", 17, true];   // arrays can hold mixed types
let empty = [];
```

Accessing Array Elements

Array elements are accessed by **index**, starting at **0**.

```
javascript
let fruits = ["apple", "banana", "mango"];

console.log(fruits[0]);  // apple
console.log(fruits[1]);  // banana
console.log(fruits[2]);  // mango
```

Array Length

The `length` property returns the number of elements in an array.

```
javascript
let fruits = ["apple", "banana", "mango"];
console.log(fruits.length);   // 3
```

Modifying Array Elements

```javascript
let fruits = ["apple", "banana", "mango"];
fruits[1] = "orange";
console.log(fruits);   // ["apple", "orange", "mango"]
```

Common Array Methods

Method	Description	Example
`push()`	Adds an element to the end	`fruits.push("kiwi")`
`pop()`	Removes the last element	`fruits.pop()`
`unshift()`	Adds an element to the beginning	`fruits.unshift("grapes")`
`shift()`	Removes the first element	`fruits.shift()`
`indexOf()`	Returns the index of an element, or -1	`fruits.indexOf("banana")`
`includes()`	Returns true if element exists	`fruits.includes("mango")`
`join()`	Joins all elements into a string	`fruits.join(", ")`
`slice()`	Returns a portion of the array	`fruits.slice(1, 3)`
`splice()`	Adds or removes elements at a position	`fruits.splice(1, 1)`
`forEach()`	Executes a function for each element	`fruits.forEach(f => console.log(f))`
`map()`	Returns a new array with each element transformed	`numbers.map(n => n * 2)`

Method	Description	Example
`filter()`	Returns a new array with elements that pass a test	`numbers.filter(n => n > 20)`

Example: Using Array Methods

```javascript
let scores = [85, 90, 78, 92, 88];

// Add a score
scores.push(95);

// Remove the last score
scores.pop();

// Filter scores above 85
let highScores = scores.filter(score => score > 85);
console.log(highScores);   // [90, 92, 88]

// Double each score
let doubled = scores.map(score => score * 2);
console.log(doubled);      // [170, 180, 156, 184, 176]
```

Iterating Over Arrays

```javascript
let fruits = ["apple", "banana", "mango"];

// Using a for loop
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}

// Using for...of
for (let fruit of fruits) {
  console.log(fruit);
```

```
}

// Using forEach
fruits.forEach(function(fruit) {
  console.log(fruit);
});
```

Multi-Dimensional Arrays

Arrays can contain other arrays.

```
javascript
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log(matrix[0][1]);   // 2
console.log(matrix[2][0]);   // 7
```

Part 6: JavaScript Objects

An **object** is a collection of **key-value pairs** that describe a single entity. While arrays are ordered lists (accessed by numeric index), objects are unordered collections (accessed by named keys). Objects are ideal for modeling real-world entities — a student, a product, a user account, or a book — because they group related properties under one name.

Creating Objects

Method 1: Object Literal (Most Common)

```javascript
let student = {
  name: "Juan Dela Cruz",
  age: 17,
  grade: "11",
  section: "STEM-A"
};
```

Method 2: `new Object()`

```javascript
let student = new Object();
student.name = "Juan Dela Cruz";
student.age = 17;
```

Accessing Object Properties

Dot notation:

```javascript
console.log(student.name);   // Juan Dela Cruz
console.log(student.age);    // 17
```

Bracket notation:

```javascript
console.log(student["name"]);   // Juan Dela Cruz
```

Bracket notation is useful when the key is stored in a variable or contains special characters.

Modifying Object Properties

```javascript
student.age = 18;
student.section = "STEM-B";
```

Adding New Properties

```javascript
student.email = "juan@example.com";
```

Deleting Properties

```javascript
delete student.section;
```

Objects with Methods

An object can contain **methods** — functions stored as properties.

```javascript
let student = {
  name: "Juan",
  age: 17,
  introduce: function() {
    return "Hi, I'm " + this.name + " and I'm " + this.age + " years old.";
  }
};

console.log(student.introduce());   // Hi, I'm Juan and I'm 17 years old.
```

The `this` keyword refers to the object the method belongs to.

Iterating Over Object Properties

```javascript
let student = { name: "Juan", age: 17, grade: "11" };

for (let key in student) {
  console.log(key + ": " + student[key]);
}
```

Arrays of Objects

A common pattern is an array where each element is an object. This is the foundation of most real-world data structures.

```javascript
let students = [
  { name: "Juan", age: 17, grade: 85 },
  { name: "Maria", age: 18, grade: 92 },
  { name: "Jose", age: 17, grade: 78 }
];

students.forEach(function(student) {
  console.log(student.name + ": " + student.grade);
});
```

Nested Objects

Objects can contain other objects.

```javascript
let order = {
  orderId: 101,
  customer: {
    name: "Maria Santos",
    address: "Manila"
  },
  items: ["laptop", "mouse", "keyboard"]
};

console.log(order.customer.name);   // Maria Santos
console.log(order.items[0]);        // laptop
```

Array vs. Object: When to Use Each

Use an Array When...	Use an Object When...
Order matters	Properties have descriptive names
Items are of the same type	The entity has distinct attributes
You need to iterate in order	You need to access by name
Example: list of grades	Example: a student record

Part 7: Document Object Model (DOM) Nodes

The **Document Object Model (DOM)** is the browser's representation of an HTML document as a tree of **nodes**. Each HTML element, text fragment, and attribute becomes a node in the tree. JavaScript can access these nodes to read, change, add, or remove content — making pages dynamic. Without the DOM, JavaScript would have no way to interact with the page the user sees.

The DOM Tree

An HTML document is organized as a tree of nodes.

```
text
document
│
└── html
    ├── head
    │   └── title
    │       └── "My Page"
    └── body
        ├── h1
        │   └── "Welcome"
        └── p
            └── "Hello, world!"
```

Each element (`html`, `head`, `body`, `h1`, `p`) is a **node**. The text inside them is also a node.

Node Types

Node Type	Description	Example
Element node	An HTML tag	`<div>`, `<p>`, `<button>`
Text node	The text inside an element	"Hello" inside a `<p>`
Attribute node	An attribute of an element	`class="container"`
Document node	The root of the DOM	`document`

Selecting Nodes

Method	Description	Example
`getElementById()`	Selects an element by its `id`	`document.getElementById("myBtn")`
`getElementsByClassName()`	Selects all elements with a class	`document.getElementsByClassName("item")`
`getElementsByTagName()`	Selects all elements of a tag	`document.getElementsByTagName("p")`
`querySelector()`	Selects the first matching CSS selector	`document.querySelector(".item")`
`querySelectorAll()`	Selects all matching CSS selectors	`document.querySelectorAll(".item")`

Example:

```
html
<h1 id="title">Original Title</h1>
<p class="description">First paragraph.</p>
<p class="description">Second paragraph.</p>

<script>
  let title = document.getElementById("title");
  let paragraphs = document.querySelectorAll(".description");
  console.log(title.textContent);          // Original Title
  console.log(paragraphs.length);          // 2
</script>
```

Reading and Changing Node Content

Property	Description
`textContent`	Gets or sets the text content (safe, no HTML parsing)
`innerHTML`	Gets or sets the HTML content (parses HTML)
`innerText`	Gets or sets the visible text

```javascript
let title = document.getElementById("title");

// Read
console.log(title.textContent);

// Change
title.textContent = "New Title";
```

Security warning: Use `textContent` instead of `innerHTML` when inserting user-provided data to prevent **Cross-Site Scripting (XSS)** attacks.

Changing Node Attributes

```javascript
let link = document.getElementById("myLink");
link.setAttribute("href", "https://example.com");
link.getAttribute("href");       // https://example.com
link.removeAttribute("href");
```

Changing Styles

```javascript
let box = document.getElementById("box");
box.style.backgroundColor = "blue";
box.style.color = "white";
box.style.padding = "20px";
```

Creating and Adding Nodes

```javascript
// Create a new element
let newParagraph = document.createElement("p");
newParagraph.textContent = "This is a new paragraph.";

// Add it to the body
document.body.appendChild(newParagraph);

// Or insert before an existing element
let container = document.getElementById("container");
container.insertBefore(newParagraph, container.firstChild);
```

Removing Nodes

```javascript
let element = document.getElementById("toRemove");
element.remove();
```

Traversing the DOM

Property	Description
`parentNode`	The parent of the current node
`childNodes`	All child nodes (including text nodes)
`children`	Only element children
`firstChild`	First child node
`lastChild`	Last child node
`nextSibling`	Next sibling node

Property	Description
`previousSibling`	Previous sibling node

Example:

```
javascript
let list = document.getElementById("myList");
console.log(list.children.length);      // number of <li> elements
console.log(list.firstChild);           // first child
```

Example: Complete DOM Interaction

```
html
<!DOCTYPE html>
<html>
<head>
  <title>DOM Example</title>
</head>
<body>
  <h1 id="title">To-Do List</h1>
  <ul id="list">
    <li>Study JavaScript</li>
    <li>Practice DOM</li>
  </ul>
  <button id="addBtn">Add Item</button>

  <script>
    let list = document.getElementById("list");
    let button = document.getElementById("addBtn");
    let counter = 2;

    button.addEventListener("click", function() {
      counter++;
      let newItem = document.createElement("li");
      newItem.textContent = "New task #" + counter;
      list.appendChild(newItem);
    });
```

```
  </script>
</body>
</html>
```

When the user clicks "Add Item," a new `<li>` node is created and appended to the list. This is the essence of dynamic DOM manipulation.

Key Terms

Term	Definition
Operator	A symbol that performs an operation on values or variables
Event	Any action or occurrence that the browser can detect and JavaScript can respond to
Event handler	A function attached to an element's event property (e.g., `onclick`)
Event listener	A function attached using `addEventListener()`, allowing multiple handlers
Event delegation	Attaching one listener to a parent element to handle events from its children
Control flow	The order in which statements execute in a program
Conditional statement	A statement (`if`, `else if`, `else`) that runs code based on a condition
Switch statement	A statement that compares an expression to multiple cases and runs the matching block
Loop	A structure that repeats a block of code multiple times
Array	An ordered collection of values accessed by numeric index
Object	A collection of key-value pairs describing an entity
Method	A function stored as a property of an object
DOM	Document Object Model; the browser's tree representation of an

Term	Definition
	HTML document
DOM node	Any element, text, or attribute in the DOM tree
`textContent`	A property that gets or sets the text content of a node
`innerHTML`	A property that gets or sets the HTML content of a node
`querySelector()`	A method that selects the first element matching a CSS selector

Review Questions

1. Explain the difference between arithmetic operators and comparison operators. Provide two examples of each.

2. Differentiate `==` and `===` in JavaScript. Why is `===` generally preferred?

3. Explain the difference between an event handler and an event listener. Provide one advantage of event listeners over event handlers.

4. Write a JavaScript snippet that changes the background color of an element with `id="box"` to blue when it is clicked. Use an event listener.

5. Differentiate `if...else` statements from `switch` statements. Provide one situation where `switch` is more appropriate than `if...else`.

6. Explain the difference between the `while` loop and the `do...while` loop. Provide one example where `do...while` is more appropriate.

7. Write a `for` loop that prints the numbers 1 to 10. Then write a `while` loop that does the same.

8. Differentiate arrays from objects. Provide one situation where an array is more appropriate and one where an object is more appropriate.

9. Study the code below and answer the questions.

```
javascript
let scores = [70, 85, 90, 65, 95];

let passing = scores.filter(function(score) {
  return score >= 75;
});

console.log(passing);
```

- (a) What does the `filter()` method do in this code?
- (b) What will be printed to the console?
- (c) How would you modify the code to also print the number of passing scores?

10. Application question: A student wants to create a simple to-do list web application where users can type a task into an input field and click an "Add" button to add the task to a list. Using the concepts from this lesson (events, DOM manipulation, arrays), describe the steps needed to build this feature. Include the event used, the DOM methods needed, and how an array could be used to keep track of tasks.

Answer Key

1. Arithmetic operators perform mathematical calculations (e.g., `5 + 3 = 8`, `10 % 3 = 1`). Comparison operators compare two values and return true or false (e.g., `5 > 3` returns `true`, `5 === "5"` returns `false`).

2. `==` compares values only and performs type coercion (`5 == "5"` is `true`). `===` compares both value and type (`5 === "5"` is `false`). `===` is preferred because it avoids unexpected bugs caused by automatic type conversion.

3. An event handler attaches a function directly to an element's event property (e.g., `button.onclick = fn`). An event listener uses `addEventListener("click", fn)`. An advantage of event listeners is that multiple listeners can be attached to the same event without overwriting each other.

4.

```javascript
let box = document.getElementById("box");
box.addEventListener("click", function() {
  box.style.backgroundColor = "blue";
});
```

5. An `if...else` statement checks conditions and can test ranges, boolean expressions, and multiple variables. A `switch` statement compares a single expression to multiple exact values. `switch` is more appropriate when checking many possible values of a single variable, such as days of the week or menu options.

6. A `while` loop checks the condition before running the body, so the body may never run. A `do...while` loop runs the body at least once, then checks the condition. `do...while` is more appropriate for input validation — you must ask the user at least once before checking if their answer is valid.

7.

```javascript
// for loop
for (let i = 1; i <= 10; i++) {
  console.log(i);
```

```
}

// while loop
let i = 1;
while (i <= 10) {
  console.log(i);
  i++;
}
```

8. Arrays are ordered collections accessed by numeric index; they are best for lists of similar items (e.g., a list of grades). Objects are key-value collections accessed by property name; they are best for describing one entity with distinct attributes (e.g., a student's name, age, and grade).

9.
(a) The `filter()` method creates a new array containing only the elements that pass the test (`score >= 75`).
(b) `[85, 90, 95]` will be printed to the console.
(c)

```
javascript
console.log(passing.length);
```

This prints `3`.

10.

- (a) Create an `<input>` field and a `<button>` with an ID.
- (b) Use `document.getElementById()` to select the input, the button, and the `<ul>` list.
- (c) Attach an **event listener** to the button for the `click` event.
- (d) Inside the listener, read `input.value`, then use `document.createElement("li")` to create a new list item, set its `textContent` to the task, and `appendChild()` it to the list.
- (e) Optionally, maintain a `tasks` array: `tasks.push(input.value)` to keep track of all tasks in memory.
- (f) Clear the input field after adding: `input.value = ""`.